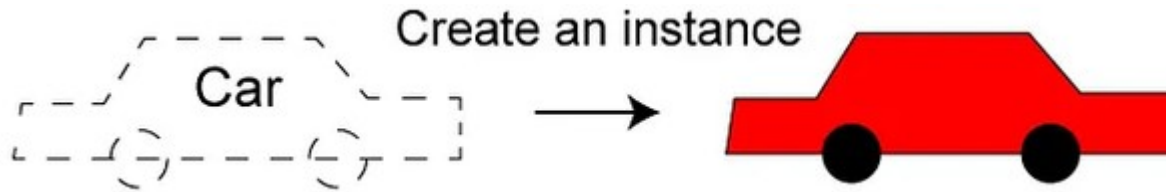


Lecture 2

Object Oriented Programming



Announcements

- ❖ No Lab on next Monday
 - Labor Day!
- ❖ Office hours are up on course website
 - T: 12pm-4pm; F: 11am-1pm
 - Schedule an appointment individually or as a group

Big Picture

- ❖ **This week:** getting up to speed with Python

- OOP & Testing

- ❖ **Next week:** start thinking algorithmically

- Greedy algorithms & empirical benchmarking
- F: Project 1 released

- ❖ **Next next week:** running time analysis

- Growth rates & Big-O
- F: Project 1 Proposal Due

Today

- ❖ Survey Object-Oriented Programming in Python
 - Constructing Classes
 - Methods & Inheritance

Classes

- ❖ Data types in Python
- ❖ Built-in: `int`, `float`, `list`, `set`, etc.

Classes

- ❖ Data types in Python
- ❖ Built-in: `int`, `float`, `list`, `set`, etc.
- ❖ An *object* is an *instance* of a class

[1,3,5,7,11,13,17]

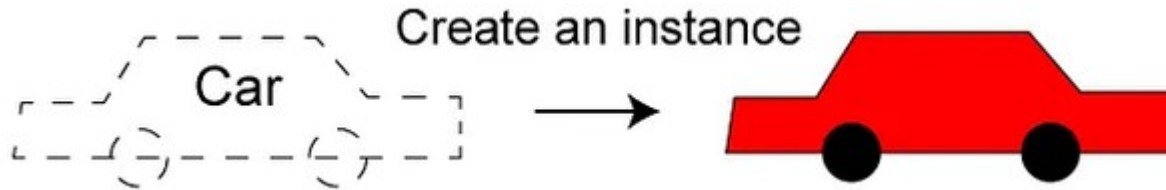
Object

list

Class

Defining a Class

- ❖ Class name
- ❖ Methods: functions callable from the class
- ❖ Attributes: variables defined for the class
- ❖ Constructor: method that creates an instance of the class



Defining a Class

```
1 class Triangle:
2     def __init__(self, angles):
3         self._sides = 3
4         self._angles = list(angles)
5         if len(self._angles) != 3 or sum(self._angles) != 180:
6             raise ValueError("Not a triangle.")
7     def sides(self):
8         return self._sides
9     def __str__(self):
10        return "I'm a triangle."
```


Defining a Class

class name



```
1 class Triangle:
2     def __init__(self, angles):
3         self._sides = 3
4         self._angles = list(angles)
5         if len(self._angles) != 3 or sum(self._angles) != 180:
6             raise ValueError("Not a triangle.")
7     def sides(self):
8         return self._sides
9     def __str__(self):
10        return "I'm a triangle."
```

Defining a Class

class name

constructor (to create instance of class)

```
1 class Triangle:
2     def __init__(self, angles):
3         self._sides = 3
4         self._angles = list(angles)
5         if len(self._angles) != 3 or sum(self._angles) != 180:
6             raise ValueError("Not a triangle.")
7     def sides(self):
8         return self._sides
9     def __str__(self):
10        return "I'm a triangle."
```

Defining a Class

The diagram illustrates the components of a Python class definition for a `Triangle` class. Annotations with arrows point to specific parts of the code:

- class name**: Points to `Triangle` in line 1.
- constructor (to create instance of class)**: Points to the `__init__` method in line 2.
- attributes**: Points to the assignments `self._sides = 3` and `self._angles = list(angles)` in lines 3 and 4.

```
1 class Triangle:
2     def __init__(self, angles):
3         self._sides = 3
4         self._angles = list(angles)
5         if len(self._angles) != 3 or sum(self._angles) != 180:
6             raise ValueError("Not a triangle.")
7     def sides(self):
8         return self._sides
9     def __str__(self):
10        return "I'm a triangle."
```

Defining a Class

```
1 class Triangle:
2     def __init__(self, angles):
3         self._sides = 3
4         self._angles = list(angles)
5         if len(self._angles) != 3 or sum(self._angles) != 180:
6             raise ValueError("Not a triangle.")
7     def sides(self):
8         return self._sides
9     def __str__(self):
10        return "I'm a triangle."
```

class name

constructor (to create instance of class)

attributes

methods

Exercise 1

What does this code output?

- a) ValueError
- b) print "I'm a triangle."
- c) print "3"
- d) Both b & c

```
1 class Triangle:
2     def __init__(self, angles):
3         self._sides = 3
4         self._angles = list(angles)
5         if len(self._angles) != 3 or sum(self._angles) != 180:
6             raise ValueError("Not a triangle.")
7     def sides(self):
8         return self._sides
9     def __str__(self):
10        return "I'm a triangle."
```

```
1 x = Triangle([40, 40, 100])
2 print(x.sides())
3 print(x)
```

Exercise 1

What does this code output?

- a) ValueError
- b) print "I'm a triangle."
- c) print "3"
- d) **Both b & c**

```
1 class Triangle:
2     def __init__(self, angles):
3         self._sides = 3
4         self._angles = list(angles)
5         if len(self._angles) != 3 or sum(self._angles) != 180:
6             raise ValueError("Not a triangle.")
7     def sides(self):
8         return self._sides
9     def __str__(self):
10        return "I'm a triangle."
```

```
1 x = Triangle([40, 40, 100])
2 print(x.sides())
3 print(x)
```

Inheritance

- ❖ Suppose we want to create a class `RightTriangle`

Inheritance

- ❖ Suppose we want to create a class `RightTriangle`
- ❖ We can *extend* the `Triangle` class

```
class RightTriangle(Triangle):  
    def __init__(self, angles):  
        Triangle.__init__(self, angles)  
        if 90 not in angles:  
            raise ValueError("Not a right triangle.")  
    def __str__(self):  
        return "I'm a right triangle."
```

parent class

call parent constructor

Exercise 2

What does this code output?

- a) ValueError
- b) print "I'm a right triangle."
- c) print "3"
- d) Both b & c

```
class RightTriangle(Triangle):  
    def __init__(self, angles):  
        Triangle.__init__(self, angles)  
        if 90 not in angles:  
            raise ValueError("Not a right triangle.")  
    def __str__(self):  
        return "I'm a right triangle."
```

```
1 x = RightTriangle([40, 40, 100])  
2 print(x.sides())  
3 print(x)
```

Exercise 2

What does this code output?

- a) **ValueError**
- b) print "I'm a right triangle."
- c) print "3"
- d) Both b & c

```
class RightTriangle(Triangle):
    def __init__(self, angles):
        Triangle.__init__(self, angles)
        if 90 not in angles:
            raise ValueError("Not a right triangle.")
    def __str__(self):
        return "I'm a right triangle."
```

```
1 x = RightTriangle([40, 40, 100])
2 print(x.sides())
3 print(x)
```

Exercise 3

What does this code output?

- a) ValueError
- b) print "I'm a right triangle."
- c) print "3"
- d) Both b & c

```
class RightTriangle(Triangle):  
    def __init__(self, angles):  
        Triangle.__init__(self, angles)  
        if 90 not in angles:  
            raise ValueError("Not a right triangle.")  
    def __str__(self):  
        return "I'm a right triangle."
```

```
1 x = RightTriangle([40, 50, 90])  
2 print(x.sides())  
3 print(x)
```

Exercise 3

What does this code output?

- a) ValueError
- b) print "I'm a right triangle."
- c) print "3"
- d) **Both b & c**

```
class RightTriangle(Triangle):  
    def __init__(self, angles):  
        Triangle.__init__(self, angles)  
        if 90 not in angles:  
            raise ValueError("Not a right triangle.")  
    def __str__(self):  
        return "I'm a right triangle."
```

```
1 x = RightTriangle([40, 50, 90])  
2 print(x.sides())  
3 print(x)
```

Abstraction

- ❖ Suppose we have classes for `Triangle` and `Square`
 - May share many methods and attributes

Abstraction

- ❖ Suppose we have classes for `Triangle` and `Square`
 - May share many methods and attributes
- ❖ Create new class called `Polygon`
 - Refactor `Triangle` and `Square` to extend `Polygon`

Abstraction

- ❖ Suppose we have classes for `Triangle` and `Square`
 - May share many methods and attributes
- ❖ Create new class called `Polygon`
 - Refactor `Triangle` and `Square` to extend `Polygon`

Good exercise if you're feeling shaky on working with classes

OOP

- ❖ What is Object-Oriented Programming?
 - Everything is an object
 - All functions are methods
 - All variables are properties
- ❖ Allows us to be very careful in specifying our code
 - Java, C++, and others follow OOP

OOP

- ❖ What is Object-Oriented Programming?
 - Everything is an object
 - All functions are methods
 - All variables are properties
- ❖ Allows us to be very careful in specifying our code
 - Java, C++, and others follow OOP
- ❖ In Python, best practice is often a hybrid approach
 - Use classes for structure and to reduce code redundancy
 - Use scripting for main process

Next Time

- ❖ Testing
- ❖ Read DSP Ch. 4